

Inference Optimisation for Large Models

Serving, Distillation, Pruning, Quantization, and TurboQuant

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

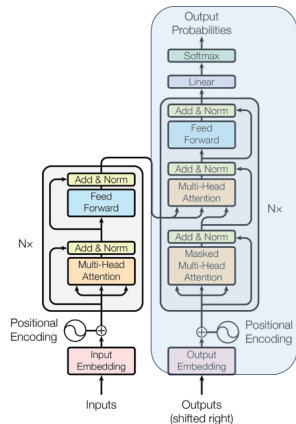
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

1. LLM inference: autoregressive decoding, prefill vs. decode, latency and throughput
2. Serving optimisations: KV caching, continuous batching, PagedAttention, and speculative decoding
3. Knowledge distillation: compression motivation and teacher–student architecture
4. Distillation training: soft targets, temperature, loss, and reasoning traces
5. Pruning: magnitude pruning, structured vs. unstructured, and 2:4 sparsity
6. Quantization: formats, granularity, weights/activations/KV cache, QAT and PTQ
7. TurboQuant: two-stage vector quantization (random rotation, then a QJL residual) for KV compression
8. Summary and references

- ▶ Most popular decoder-only LLMs (e.g., GPT-3) are pretrained for **causal language modeling**: predicting the next token in a sequence.
- ▶ At inference, LLMs receive a series of **tokens** as input and generate the following tokens **autoregressively** until a stopping criterion is met (e.g., token limit, stop word, or special <end> token).
- ▶ **Tokens** are about 4 English characters each.
- ▶ The inference process has **two phases**: **prefill** (process prompt in parallel) and **decode** (step-by-step token generation).



Highlighted: the stack a decoder-only LLM keeps. Vaswani et al., "Attention Is All You Need," [arXiv:1706.03762](https://arxiv.org/abs/1706.03762),

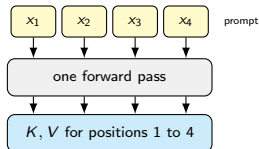
Figure 1.

► Prefill phase (input processing):

- All input tokens are processed together to compute **keys** and **values** (attention states) for the prompt.
- The entire input sequence is known in advance, enabling a highly parallel **matrix-matrix** computation that fully uses the GPU's throughput.

► Token throughput caveat:

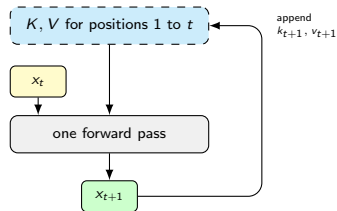
- Different LLMs may use different **tokenizers**, so “tokens per second” is not always comparable across models, as token-to-character mappings vary.



All prompt positions are computed in a single pass.

► Decode phase (generating outputs):

- Output tokens are generated **one at a time**, each conditioned on all previously generated tokens.
- Computation now becomes a **matrix-vector** operation, less parallelizable and typically **memory-bound**: moving weights/activations dominates speed, not raw compute.
- Optimising decode is crucial: efficient attention, key/value caching, and memory management are ongoing research topics.

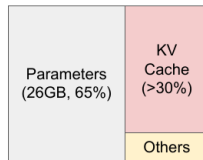


One position per pass; the cache grows every step.

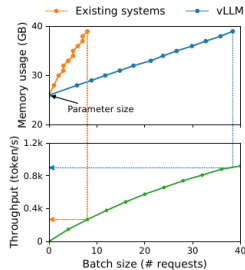
- ▶ **Training** is often one-off; **inference** is repeated at scale (cost, latency, energy).
- ▶ Large language models are **memory-bound** and **autoregressive**: each new token revisits all previous tokens.
- ▶ Goals: lower **latency** (interactive UX), higher **throughput** (serving many users), smaller **memory** (long contexts, edge).

- ▶ **Time to first token (TTFT):** how long until the first output token appears; set by the prefill phase, which is compute-bound.
- ▶ **Time per output token (TPOT):** the gap between subsequent tokens; set by the decode phase, which is memory-bound.
- ▶ Request latency $\approx \text{TTFT} + \text{TPOT} \times \text{number of generated tokens}$.
- ▶ **Throughput:** total tokens per second across all requests. Batching raises throughput but can hurt per-request latency; serving systems balance the two.

- ▶ As covered earlier, **KV caching** keeps the key and value tensors of past positions in GPU memory so decode does not recompute them, at the price of memory that grows with layers, batch size, and context length.
- ▶ That memory is the binding constraint in serving: after the weights are loaded, whatever is left is what caps how many requests can run at once.
- ▶ Attention variants such as MQA and GQA shrink the cache by sharing key and value heads. The rest of this section attacks the same budget from two other directions: **how the cache is allocated** (paging) and **how many bits each entry costs** (quantization).



NVIDIA A100 40GB

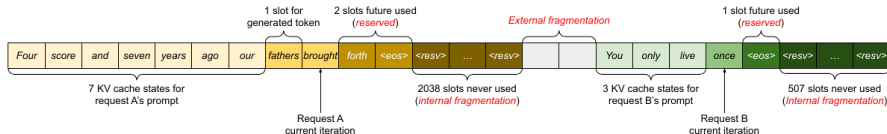


Serving a 13B model on an NVIDIA A100 (40 GB): weights take 65% of memory, the KV cache over 30%. Kwon et al.,

[arXiv:2309.06180](https://arxiv.org/abs/2309.06180), Figure 1.

- ▶ A single decode stream is a matrix-vector workload that leaves most of the GPU idle; batching many requests turns it back into matrix-matrix work.
- ▶ **Static batching:** requests are grouped up front and the whole batch runs until its longest member finishes; short requests hold their slot while doing nothing.
- ▶ **Continuous batching:** scheduling happens per decode step; finished sequences leave the batch immediately and waiting requests join mid-flight. Introduced as **iteration-level scheduling** by Orca (Yu et al., 2022) and later popularised under the name continuous batching.
- ▶ Standard in modern serving stacks (vLLM, TensorRT-LLM, TGI): large throughput gains with no change to model outputs.

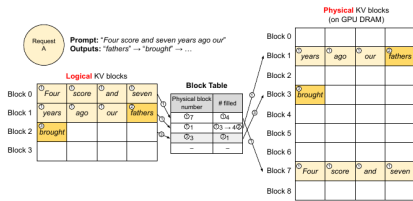
- ▶ Early serving systems stored each request's KV cache as one contiguous tensor, allocated up front for the maximum possible sequence length.
- ▶ Three kinds of waste follow: reserved slots not yet used, internal fragmentation (slots never used because the request ends early), and external fragmentation between buffers.
- ▶ Measured effective KV memory utilisation in such systems can drop to about 20%.



Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," [arXiv:2309.06180](https://arxiv.org/abs/2309.06180),

Figure 3.

- ▶ Borrow virtual memory from operating systems: split each request's KV cache into fixed-size blocks and allocate blocks on demand.
- ▶ A block table maps logical positions to physical blocks, so the cache no longer needs to be contiguous; waste is limited to the last partially filled block, and blocks can be shared across sequences (parallel sampling, shared prefixes).
- ▶ vLLM built on this achieves $2\text{--}4\times$ the throughput of prior serving systems at the same latency.



Block table translation. Kwon et al., [arXiv:2309.06180](https://arxiv.org/abs/2309.06180), Figure 6.

- ▶ Decode is memory-bound: for the large model, scoring K proposed tokens in one forward pass costs about the same as generating a single token.
- ▶ A small **draft model** proposes K tokens autoregressively; the large **target model** then evaluates all K positions in parallel.
- ▶ Accept draft tokens while they are consistent with the target distribution; at the first rejection, resample that token from an adjusted distribution and discard the rest.
- ▶ The output distribution is provably identical to decoding with the target model alone; the speed comes from accepting several tokens per target pass.
- ▶ Reported speedups with no change to the output distribution: $2\text{--}3\times$ on T5-XXL (Leviathan et al., 2023) and $2\text{--}2.5\times$ on Chinchilla 70B (Chen et al., 2023).

- ▶ Write q for the target model's next-token distribution and p for the draft model's. For each proposed token $\tilde{x}$, draw $r \sim U[0, 1]$ and **accept** it when

$$r < \min\left(1, \frac{q(\tilde{x} \mid x_{1:n})}{p(\tilde{x} \mid x_{1:n})}\right).$$

- ▶ So a token the target likes at least as much as the draft does is always accepted; one the draft over-proposed is accepted only in proportion q/p .
- ▶ On the first rejection, the remaining proposals are discarded and that position is resampled from the **residual** distribution $(q(x \mid x_{1:n}) - p(x \mid x_{1:n}))_+$, renormalised, where $(f)_+ = \max(0, f)$. This is what makes the accept/reject step exact rather than an approximation.
- ▶ If all K proposals are accepted, one extra token is sampled from the target's own distribution, so a single target pass can yield up to $K + 1$ tokens.

Modified rejection sampling, Algorithm 2 and Theorem 1 of Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," [arXiv:2302.01318](https://arxiv.org/abs/2302.01318).

```
[START] japan ' s benchmark bond n
[START] japan ' s benchmark nikkei 22 5
[START] japan ' s benchmark nikkei 225 index rose 22 6
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in tokyo late
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in late morning trading . [END]
```

- ▶ Each line is one iteration: green tokens were proposed by the draft model and accepted, red were rejected, and blue are the target model's corrections.
- ▶ This 38-token sentence needed only 9 serial passes of the target model.
- ▶ Gains are largest when the draft agrees with the target on the easy tokens, which is most of them.

Leviathan, Kalman & Matias, "Fast Inference from Transformers via Speculative Decoding," [arXiv:2211.17192](https://arxiv.org/abs/2211.17192), Figure 1.

- ▶ **Kernel fusion & IO-aware attention** (e.g. FlashAttention-style): reduce HBM traffic.
- ▶ **Prefix caching**: reuse the KV cache of shared prompt prefixes (system prompts, few-shot examples) across requests.
- ▶ **Weight sharing / low-rank adapters (LoRA)**: smaller footprints where applicable.
- ▶ **Approximate attention** (long contexts): trade accuracy for sub-quadratic cost.

Model Optimisation Techniques



Harvard IACS AC215, Lecture 11, slide 8.

Problem:

- ▶ During **training**, a model does not have to operate in real time and does not necessarily face restrictions on computational resources. The primary goal is to extract as much structure from the given data as possible.
- ▶ But latency and resource consumption do become of concern if the model is to be deployed for **inference**.

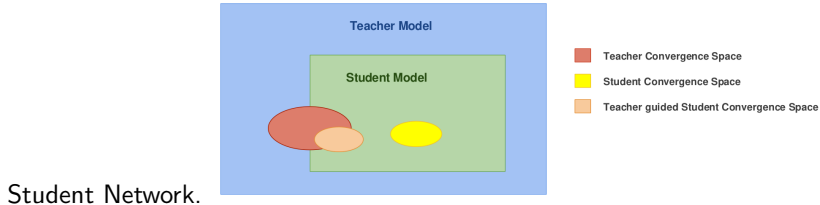
So what? We must develop ways to compress models for inference.

Idea:

- ▶ In 2006, Bucila et al. showed that it was possible to transfer knowledge from a large trained model (or ensemble of models) to a smaller model for deployment by **training it to mimic the larger model's output**.
- ▶ In 2014 Hinton et al. generalized the process and gave the name **Distillation**.

Main idea of distillation is that **training and inference are 2 different tasks**; thus a **different model should be used**.

Assumption: If we can achieve similar convergence using a **smaller network**, then the convergence space of the Teacher Network should **overlap** with the solution space of the



Teacher/Student convergence regions. Harvard AC215, L11.

While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

This is a Dog $\left. \begin{array}{c} \text{one-hot} \end{array} \right\} \begin{array}{c} 0 \\ 0 \\ 1 \\ 0 \end{array} \longrightarrow \text{Dog}$



Photo: Harvard IACS AC215, Lecture 11, p. 14.

While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

This is also a Dog

one-hot $\left\{ \begin{array}{c} 0 \\ 0 \\ 1 \\ 0 \end{array} \right\} \longrightarrow \text{Dog}$



Photo: Harvard IACS AC215, Lecture 11, p. 15.

While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

This is a Cat one-hot $\left\{ \begin{array}{l} 0 \\ 0 \\ 0 \\ 1 \end{array} \right\} \longrightarrow \text{Cat}$



Photo: Harvard IACS AC215, Lecture 11, p. 17.

What about this example?

This is a ...

$$\begin{pmatrix} 0 \\ 0 \\ 0.5 \\ 0.5 \end{pmatrix}$$



Photo: Harvard IACS AC215, Lecture 11, p. 19.

In the real world, the distinction is never that clear. Ideally, the **training labels must be soft**, to learn about the nuanced relations across the elements of the dataset.



Photo: Harvard IACS AC215, Lecture 11, **p. 20**.

The teacher shows the student the **relations** between the different classes, based on what it **learned** during its training stage.

From a dataset containing the original categories, the teacher outputs the **normalized probabilities** for each example.

	cow	car	dog	cat
Original labels	0	1	0	0
Probabilities	10^{-6}	0.9	0.1	10^{-9}

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*

Distillation: Teacher – Student (Softening with Temperature)

To enhance the student's learning, the teacher **softens** its output z even more.

To do this, it introduces the softmax function with temperature T :

$$q_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

The softened row is the same logits z pushed through this softmax at $T = 5$; like any softmax output, it still sums to 1.

“Soft” targets allow the student to learn the teacher's implicit knowledge about class similarities.

	cow	car	dog	cat
Original labels	0	1	0	0
Softmax probabilities	10^{-6}	0.9	0.1	10^{-9}
Softened, $T=5$	0.04	0.58	0.37	0.01

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean,
Distilling the Knowledge in a Neural Network

A high value of T will dilute the information contained in the probabilities.

- ▶ The standard softmax is recovered for $T=1$.
- ▶ For $T>1$, the probabilities are softened. Typical values range between 2 and 5.
- ▶ For $T<1$, it approaches to the original one-hot labels.

	cow	car	dog	cat
Original labels	0	1	0	0
Softmax probabilities	10^{-6}	0.9	0.1	10^{-9}
Softened, $T=5$	0.04	0.58	0.37	0.01

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean,
Distilling the Knowledge in a Neural Network

- ▶ The student is trained to both:
 - Solve the original task (matching the hard/true labels).
 - Replicate the softened probabilities (soft labels) from the teacher.
- ▶ This allows the student to learn not only the correct answer, but also the teacher's representation of class similarities.

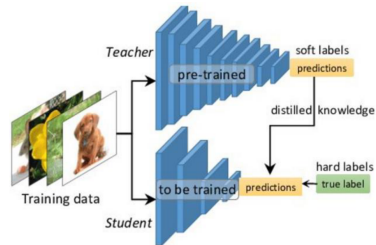


Figure: Harvard IACS AC215, Lecture 11, p. 25; credited there to Hinton, Vinyals & Dean, *Dark Knowledge*.

Scenario 1:

Train on the same data as the teacher model.

(DistilBERT was trained on this example.)

Scenario 2:

Train on a reduced dataset. This is usually done when you have the teacher model but not the original dataset or resources.

It minimizes the sum of two different cross-entropy functions:

- ▶ One involving the original hard labels obtained using a softmax with $T = 1$.
- ▶ One involving the softened probabilities with $T > 1$.
- ▶ The gradients of the soft-target term scale as $1/T^2$, so in practice that term is multiplied by T^2 . Without it, raising T would quietly shrink the distillation signal relative to the hard-label one.

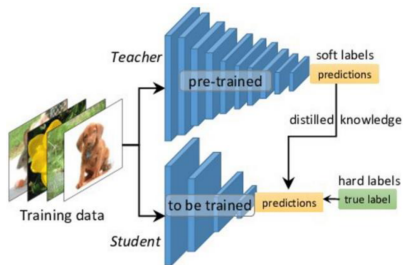


Figure: Harvard IACS AC215, Lecture 11, [p. 25](#);
credited there to Hinton, Vinyals & Dean, *Dark Knowledge*.

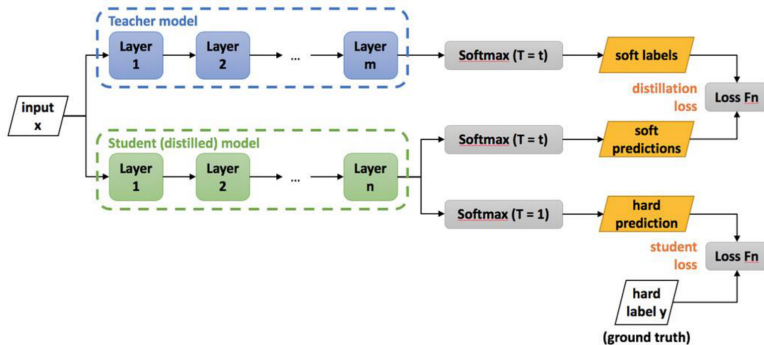


Figure: Harvard IACS AC215, Lecture 11, p. 28. Temperature and loss scheme: Hinton, Vinyals & Dean, *Distilling the Knowledge in a Neural Network*.

$$L = \lambda L_{\text{student}} + (1 - \lambda) L_{\text{distillation}}$$

$$L_{\text{Distillation}} = - \sum_{k=1}^K p_T^k(x, T) \log(q_S^k(x, T))$$

In our example, for the example n in the dataset:

$$p_T(x, T) = [0.04, 0.58, 0.37, 0.01]$$

And our predictions

$$q_S(x, T) = [0.1, 0.45, 0.30, 0.15]$$

The resulting loss value for this example is:

$$- \left[0.04 \log(0.1) + 0.58 \log(0.45) + 0.37 \log(0.30) + 0.01 \log(0.15) \right] \approx 1.02$$

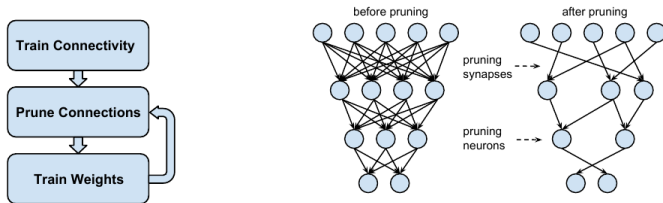
- ▶ Classic distillation transfers the teacher's **output distribution**: the student matches softened logits, one prediction at a time.
- ▶ For reasoning models the valuable signal is the **trace**: the chain of intermediate steps the teacher produces before its answer.
- ▶ **Recipe**: generate many problems' worth of full reasoning traces with a large reasoning model, filter for correct answers, then fine-tune the small model on the traces with plain supervised learning. No logits, no RL. The filter-on-correct-answers step is the idea behind STaR (Zelikman et al., 2022).
- ▶ DeepSeek distilled R1 traces into Qwen and Llama models from 1.5B to 70B this way, using 800k curated samples and supervised fine-tuning only; the distilled students beat much larger non-reasoning baselines on math and code benchmarks.
- ▶ The trade: trace distillation is far cheaper than running RL on the small model, but the student inherits the teacher's reasoning style, including its mistakes.

DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,"

[arXiv:2501.12948](https://arxiv.org/abs/2501.12948).

Pruning

- ▶ Trained networks are heavily over-parameterised: many weights contribute almost nothing to the output.
- ▶ **Magnitude pruning:** remove the connections with the smallest absolute weights, then retrain the survivors to recover accuracy; repeat iteratively.
- ▶ Han et al. (2015) cut AlexNet weights by $9\times$ and VGG-16 by $13\times$ with no accuracy loss.



Han et al., "Learning both Weights and Connections for Efficient Neural Networks," [arXiv:1506.02626](https://arxiv.org/abs/1506.02626), Figures 2–3.

- ▶ **Unstructured pruning** removes individual weights: highest compression, but the result is a sparse matrix that standard GPU kernels cannot run faster.
- ▶ **Structured pruning** removes whole neurons, attention heads, or layers: less aggressive, but leaves a smaller dense model that is faster on any hardware.
- ▶ **2:4 semi-structured sparsity**: keep 2 weights in every group of 4; NVIDIA tensor cores (Ampere onward) run it at up to $2\times$ math throughput (Mishra et al., 2021).
- ▶ For LLMs, one-shot post-training methods (SparseGPT, Wanda) reach about 50% sparsity with modest quality loss and no retraining.
- ▶ Pruning composes with distillation and quantization; production LLM serving still leans mostly on quantization, with pruning gaining ground through 2:4 hardware support.

Quantization

Quantization is a **key** compression technique for **LLMs** and **large transformers** to fit on **constrained hardware**.

- ▶ **Goal:** run larger models on **constrained** hardware with acceptable accuracy.
- ▶ Lowering precision on **weights and/or activations** (e.g. FP32/FP16 $\rightarrow$ FP8) cuts **memory**, speeds **inference**, and reduces **energy**, at some accuracy cost; the right trade-off is use-case dependent.

- ▶ **Weights:** straightforward win on footprint; Llama 2 7B at FP16/BF16 $\approx$ **14 GB** $\rightarrow$ FP8 weights $\approx$ **7 GB** (blog numbers).
- ▶ **Activations:** intermediate layer outputs; quantizing compute can exploit **tensor cores** at lower bit width for more throughput.
- ▶ **KV cache** (decoder-only): grows with context, layers, and heads, often **several GB** at long context (e.g. 4096 tokens) alongside weights.
- ▶ **Deployment patterns:** weight-only INT8/INT4 for maximum weight memory savings (activations may stay higher precision); **KV cache quantization** when long context dominates.

- ▶ Floating point formats use their bits to store three parts: **sign** (positive/negative), **exponent** (size/range), and **mantissa** (precision of the value). For example, FP8 E4M3 uses 4 bits for the exponent and 3 bits for the mantissa.
- ▶ With e exponent bits and m mantissa bits, a normal value is $x = (-1)^s \cdot 2^{E-\beta} \cdot (1 + M/2^m)$, where E and M are the stored exponent and mantissa fields and $\beta = 2^{e-1} - 1$ is the **exponent bias**. The bias is what allows exponents below zero, and the leading 1 is implicit rather than stored.
- ▶ Typical formats: FP32, FP16, BF16, FP8, and FP4. Main difference is how much range and precision you get: higher precision means more accurate numbers, but also more memory and compute.



Figure: FP16, BF16, FP8 and FP4 layouts with their ranges.
Reference: Wang & Spindler, NVIDIA Technical Blog (2025),
Figure 1.

- ▶ Quantization maps $x \in [\alpha, \beta]$ to a low-precision $x_q \in [\alpha_q, \beta_q]$ using two parameters: the **scale** s , a positive float setting the step size, and the **zero-point** z , which fixes where real zero lands:

$$x_q = \text{clip}(\text{round}(x/s + z), \alpha_q, \beta_q)$$

and dequantizing gives $x \approx s(x_q - z)$.

- ▶ Mapping zero *exactly* matters because operators such as padding insert real zeros; a zero that drifts becomes a systematic error.
- ▶ **Symmetric** quantization fixes $z=0$, leaving $x_q = \text{clip}(\text{round}(x/s), \alpha_q, \beta_q)$ and $x \approx s x_q$: fewer additions at run time, and the scheme TensorRT and Model Optimizer use.
- ▶ **Rounding** and **clipping** both lose information, and dequantizing does not recover it.



Figure: affine vs. symmetric quantization. Reference: Wang & Spindler, NVIDIA Technical Blog (2025), Figure 2.

- ▶ How many (s, z) pairs to keep, and over what span of the tensor each one applies:
- ▶ **Per-tensor / per-layer:** one (s, z) for the whole tensor: simplest, smallest metadata, can mis-handle heterogeneous ranges.
- ▶ **Per-channel:** separate parameters per channel (e.g. conv channels): isolates **outliers** to their channel.
- ▶ **Per-block / per-group:** parameters per sub-block: finer control when mass varies inside a tensor.
- ▶ Finer granularity tracks the data better but stores more scales, so the choice is a memory-versus-accuracy trade like the bit-width itself.

Per-Tensor:



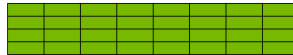
All elements share the same scale and zero-point

Per-Channel:



Each channel (horizontal strip) has its own scale and zero-point

Per-Group:



Each group (small block) has its own scale and zero-point

Figure: per-tensor, per-channel and per-block granularity. Reference:
Wang & Spindler, NVIDIA Technical Blog (2025), Figure 4.

- ▶ Bake quantization into training: forward/backward see **fake-quant** (quantize then dequantize) while master weights stay high precision for optimiser steps.
- ▶ Weights adapt to **rounding/clipping** error; fake-quant modules often **frozen** while weights are fine-tuned.
- ▶ One obstacle: $\text{round}(\cdot)$ has zero gradient almost everywhere, so a naive backward pass would stop learning. QAT uses the **straight-through estimator**: the rounding step is treated as the identity on the backward pass, passing the incoming gradient through unchanged.
- ▶ The result costs training compute that post-training methods do not, and is worth it mainly at the low bit-widths where post-training quantization starts to lose accuracy.

- ▶ **Weight-only quantization:** Quantize the model's trained weights to lower precision using a calculated scale (with or without a zero-point). Since weights are fixed after training, no input data is needed.
- ▶ **Weights and activations quantization:** Requires representative input data to measure activation statistics (by running the model), which are then used to determine quantization parameters (scale and zero-point) through a process called calibration.

- ▶ **Static quantization:** Calibration uses a dataset to determine quantization parameters once; these are kept fixed for all future inference.
- ▶ **Dynamic quantization:** Quantization parameters are computed on the fly during inference for each input, so no calibration set is needed but more runtime overhead is incurred.

TurboQuant

- ▶ **Vector quantization (VQ):** compress high-dimensional vectors while controlling **distortion** of their geometry.
 - ▶ Enhances vector search and KV cache compression.
- ▶ Prior practical schemes often fall **short of optimal** distortion–rate scaling.
- ▶ TurboQuant shrinks what inference *stores* rather than the model itself: the **KV cache** during decoding, and the **embedding vectors** of a search index. Model weights are not its target.
- ▶ Reported quality: neutral at **3.5 bits** per channel for the KV cache, with marginal degradation at 2.5 bits.
- ▶ Applies to any model at inference time with no offline preparation or calibration data.

- ▶ For a budget of b bits per coordinate, no quantizer whatsoever can push the mean squared error below a fixed floor; the paper proves that floor is of order $1/4^b$.
- ▶ TurboQuant’s own distortion is shown to sit within a constant factor of at most $\sqrt{3}\pi/2 \approx 2.7$ of that floor, simultaneously for every bit-width and every dimension.
- ▶ The guarantee is worst-case and **data-oblivious**: it holds without a calibration set and without a codebook fitted to the data, which is what allows the quantizer to run online.
- ▶ So “near-optimal” is a proved bound on MSE distortion, not a benchmark score.

Zandieh, Daliri, Hadian & Mirrokni, “TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate,”
[arXiv:2504.19874](#).

► TurboQuant works in **two stages**:

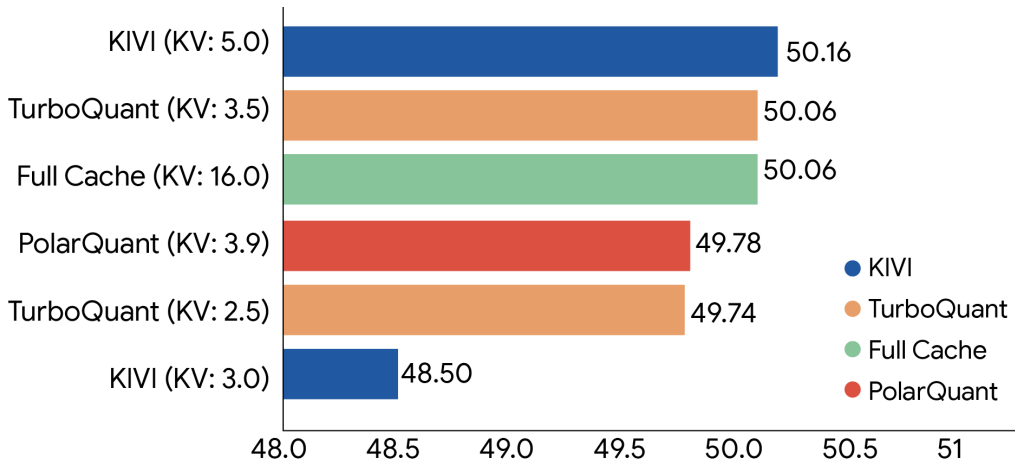
- **Stage 1: random rotation, then scalar quantization.** Randomly rotating a vector spreads its mass evenly, so its coordinates become nearly independent and each follows the same known distribution. That lets a single **MSE-optimal scalar quantizer** be applied coordinate by coordinate, spending most of the bit budget well without any codebook fitted to the data.
- **Stage 2: removing the inner-product bias with QJL.** An MSE-optimal quantizer is biased when its output is used inside a *dot product*, which is exactly what attention computes. TurboQuant spends one remaining bit on a **QJL (Quantized Johnson–Lindenstrauss)** transform of the residual, which cancels that bias and makes the estimated attention scores unbiased.

► **PolarQuant** is a companion method from the same group, not a stage of TurboQuant: it quantizes vectors through a polar transformation and serves here as one of the baselines TurboQuant is measured against.

Zandieh & Mirrokni, “TurboQuant: Redefining AI efficiency with extreme compression,” Google Research Blog (2026).

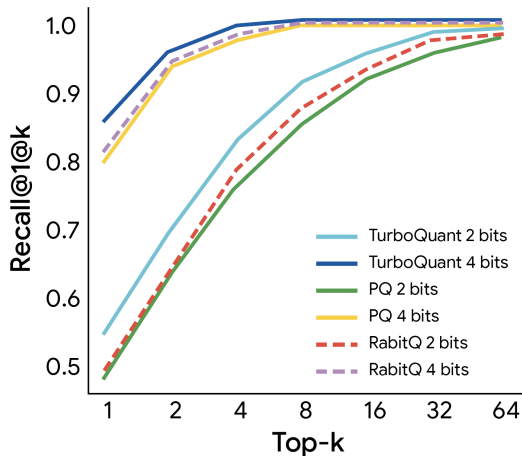
[Article.](#)

TurboQuant: Experiments and Results



KV cache compression on the LongBench benchmark (QA, code generation, summarization) with Llama-3.1-8B-Instruct:

TurboQuant vs. PolarQuant and KIVI; bit-widths in brackets. Reference: Google Research Blog.



1@k recall on GloVe ($d=200$): TurboQuant vs. Product Quantization (PQ) and RaBitQ. Reference: Google Research Blog.

- ▶ **Inference optimisation lowers latency, increases throughput, and reduces memory.**
- ▶ **KV caching** avoids recomputing past keys/values but grows memory with layers, batch, and length, motivating KV quantization, grouped attention, and paging.
- ▶ **Serving:** continuous batching keeps the GPU full; PagedAttention removes KV cache fragmentation; speculative decoding buys roughly 2–3× decode speed while preserving the target model's output distribution.
- ▶ **Distillation:** train a compact **student** to mimic a **teacher** for deployment; **soft targets** at raised **temperature** carry knowledge beyond one-hot labels.
- ▶ **Pruning:** remove low-value weights (unstructured, structured, or 2:4 semi-structured) and retrain; composes with the other compression techniques.
- ▶ **Quantization:** shrink **weights**, **activations**, and the **KV cache** via lower-bit formats.
- ▶ **TurboQuant:** random rotation plus a per-coordinate scalar quantizer, with one bit of QJL residual to keep attention scores unbiased. It compresses the **KV cache** and search-index vectors rather than model weights, with a proved near-optimal distortion bound and no calibration data.

1. Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. [arXiv:1706.03762](#)
2. Bucila, Caruana, Niculescu-Mizil, “Model Compression,” KDD 2006, pp. 535–541. [PDF](#)
3. Hinton, Vinyals, Dean, “Distilling the Knowledge in a Neural Network,” NIPS 2014 Deep Learning Workshop. [arXiv:1503.02531](#)
4. Hinton, Vinyals, Dean, “Dark Knowledge,” talk slides, 2014. [PDF](#)
5. Sanh, Debut, Chaumond, Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing, NeurIPS 2019. [arXiv:1910.01108](#)
6. Zelikman et al., “STaR: Bootstrapping Reasoning With Reasoning,” NeurIPS 2022. [arXiv:2203.14465](#)
7. DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025. [arXiv:2501.12948](#)
8. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” CVPR 2018. [arXiv:1712.05877](#)
9. Leviathan, Kalman, Matias, “Fast Inference from Transformers via Speculative Decoding,” ICML 2023. [arXiv:2211.17192](#)
10. Chen et al., “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. [arXiv:2302.01318](#)

11. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP 2023. [arXiv:2309.06180](#)
12. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” OSDI 2022. [Paper](#) (iteration-level scheduling)
13. Han et al., “Learning both Weights and Connections for Efficient Neural Networks,” NeurIPS 2015. [arXiv:1506.02626](#)
14. Mishra et al., “Accelerating Sparse Deep Neural Networks,” 2021. [arXiv:2104.08378](#) (2:4 sparsity)
15. Frantar & Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot,” ICML 2023. [arXiv:2301.00774](#)
16. Sun et al., “A Simple and Effective Pruning Approach for Large Language Models,” ICLR 2024 (Wanda). [arXiv:2306.11695](#)
17. Verma, Vaidya, “Mastering LLM Techniques: Inference Optimization,” NVIDIA Developer Blog, 17 Nov. 2023. [Article](#)
18. Wang, Spindler, “Model Quantization: Concepts, Methods, and Why It Matters,” NVIDIA Developer Blog, 24 Nov. 2025. [Article](#)
19. Zandieh, Daliri, Hadian, Mirrokni, “TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate,” ICLR 2026. [arXiv:2504.19874](#)

20. Zandieh, Mirrokni, “TurboQuant: Redefining AI efficiency with extreme compression,” Google Research Blog, 24 Mar. 2026. [Article](#)
21. Zandieh, Daliri, Han, “QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead,” AAAI 2025. [arXiv:2406.03482](#)
22. Han, Kacham, Karbasi, Mirrokni, Zandieh, “PolarQuant: Vector Quantization with Polar Transformation,” AISTATS 2026. [arXiv:2502.02617](#)
23. Gao & Long, “RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search,” SIGMOD 2024. [arXiv:2405.12497](#)
24. M. Naik, *Introduction*, Lecture 1, CIS 7000: *Large Language Models*, University of Pennsylvania, Fall 2024. [PDF](#)
25. Harvard University IACS, AC215: *MLOps, LLMops & AIOps: Productionizing AI Systems* (Fall 2024), Lecture 11: *Models Compression Techniques* (slides 7–34 used for the distillation figures). [PDF](#)
26. T. Chen & Z. Jia, *LLM Finetuning Techniques*, Lecture 17, CMU 15-779: *Advanced Topics in Machine Learning Systems (LLM Edition)*, Fall 2025. [PDF](#)